

I/O trong Go

Hướng Dẫn Toàn Diện

Từ interface cơ bản đến HTTP, Network, Encoding và Custom I/O

1. Kiến Trúc Tổng Quan

Go tổ chức toàn bộ I/O xung quanh hai interface cốt lõi: `io.Reader` và `io.Writer`. Mọi nguồn dữ liệu (file, network, memory, HTTP body) đều implement các interface này, cho phép kết hợp tự do không phụ thuộc vào kiểu cụ thể.

2. Toàn Bộ Interface I/O trong Go

2.1 Primitive Interfaces

`io.Reader`

```
Read(p []byte) (n int, err error)
```

Interface cơ bản nhất. Đọc tối đa `len(p)` bytes vào `p`. Trả về số bytes đọc được. Trả về `io.EOF` khi hết dữ liệu.

`io.Writer`

```
Write(p []byte) (n int, err error)
```

Ghi `len(p)` bytes từ `p` vào stream. Phải trả về lỗi nếu `n < len(p)`. Không được thay đổi nội dung slice `p`.

io.Closer`Close() error`

Đóng resource. Sau khi Close, mọi thao tác Read/Write đều phải trả lỗi.

io.Seeker`Seek(offset int64, whence int) (int64, error)`

Di chuyển con trỏ đọc/ghi. whence: io.SeekStart (0), io.SeekCurrent (1), io.SeekEnd (2). Trả về vị trí mới.

2.2 Composed Interfaces

Go khuyến khích ghép các interface nhỏ thành interface lớn hơn khi cần:

Interface	Ghép từ	Ứng dụng điển hình
io.ReadWriter	Reader + Writer	bytes.Buffer, net.Conn, *os.File
io.ReadCloser	Reader + Closer	http.Request.Body, http.Response.Body
io.WriteCloser	Writer + Closer	gzip.Writer, os.Pipe write-end
io.ReadWriteCloser	Reader + Writer + Closer	net.Conn, os.File
io.ReadWriteSeeker	Reader + Writer + Seeker	*os.File
io.ReadSeeker	Reader + Seeker	strings.Reader, bytes.Reader, *os.File

2.3 Advanced Interfaces

io.ReaderAt`ReadAt(p []byte, off int64) (n int, err error)`

Đọc tại offset chỉ định mà KHÔNG di chuyển con trỏ file. Thread-safe — nhiều goroutine có thể ReadAt đồng thời. Dùng cho random access.

io.WriterAt`WriteAt(p []byte, off int64) (n int, err error)`

Ghi tại offset chỉ định mà không di chuyển con trỏ. Implement bởi *os.File.

io.ReaderFrom

```
ReadFrom(r Reader) (n int64, err error)
```

Cho phép `io.Copy` tối ưu hóa: khi `dst` implement `ReaderFrom`, `io.Copy` gọi `dst.ReadFrom(src)` thay vì dùng buffer trung gian. Ví dụ: copy file sang TCP dùng `sendfile()` syscall (zero-copy).

io.WriterTo

```
WriteTo(w Writer) (n int64, err error)
```

Đối xứng với `ReaderFrom`. Cho phép `src` kiểm soát cách ghi vào `dst`.

2.4 Byte-level Interfaces

Interface	Method	Mô tả
<code>io.ByteReader</code>	<code>ReadByte()</code> (byte, error)	Đọc 1 byte. Hiệu quả hơn <code>Read(p[:1])</code>
<code>io.ByteWriter</code>	<code>WriteByte(c byte) error</code>	Ghi 1 byte
<code>io.ByteScanner</code>	<code>ByteReader</code> + <code>UnreadByte()</code>	Đọc 1 byte + có thể 'trả lại' byte vừa đọc
<code>io.RuneReader</code>	<code>ReadRune()</code> (r rune, size int, err error)	Đọc 1 rune UTF-8
<code>io.RuneScanner</code>	<code>RuneReader</code> + <code>UnreadRune()</code>	Đọc rune + có thể trả lại
<code>io.StringWriter</code>	<code>WriteString(s string) (n int, err error)</code>	Ghi string không cần chuyển sang <code>[]byte</code>

2.5 io/fs — Virtual Filesystem Interface

```
// Embed files vào binary lúc compile time
//go:embed templates/* static/css static/js/app.js
var content embed.FS

// Đọc file embedded
data, _ := content.ReadFile("static/index.html")

// Walk toàn bộ embedded FS
fs.WalkDir(content, ".", func(path string, d fs.DirEntry, err error) error {
    fmt.Println(path)
    return nil
})

// HTTP file server từ embedded FS
http.Handle("/static/", http.StripPrefix("/static/", http.FileServerFS(content)))

// Sub-filesystem
subFS, _ := fs.Sub(content, "static")
http.Handle("/", http.FileServerFS(subFS))
```

```
// Implement FS cho testing
type mockFS map[string]string
func (m mockFS) Open(name string) (fs.File, error) {
    content, ok := m[name]
    if !ok {
        return nil, &fs.PathError{Op: "open", Path: name, Err: fs.ErrNotExist}
    }
    return &mockFile{Reader: strings.NewReader(content)}, nil
}
```

3. Package os — File I/O

3.1 Mở và Đọc File

```
// Mở file chỉ để đọc
f, err := os.Open("data.txt")
if err != nil {
    log.Fatal(err)
}
defer f.Close() // LUÔN defer Close ngay sau Open

// Đọc toàn bộ file vào []byte
data, err := io.ReadAll(f)

// Shortcut: đọc toàn bộ file (không cần Open/Close)
data, err = os.ReadFile("config.json")

// Ghi toàn bộ file
err = os.WriteFile("output.txt", data, 0644)
```

3.2 Tạo và Ghi File

```
// os.Create: tạo mới hoặc truncate nếu đã tồn tại
f, err := os.Create("output.txt")
if err != nil { log.Fatal(err) }
defer f.Close()

f.WriteString("Hello, Go!\n")
f.Write([]byte("Raw bytes\n"))
fmt.Fprintf(f, "Formatted: %d\n", 42)
```

3.3 OpenFile — Kiểm soát chi tiết

```
// Append vào file log
f, err := os.OpenFile("app.log",
    os.O_APPEND|os.O_CREATE|os.O_WRONLY,
    0644, // permission: rw-r--r--
)
```

Flag	Ý nghĩa
os.O_RDONLY	Chỉ đọc (mặc định)
os.O_WRONLY	Chỉ ghi
os.O_RDWR	Đọc và ghi
os.O_APPEND	Ghi nối vào cuối file
os.O_CREATE	Tạo mới nếu chưa tồn tại

os.O_TRUNC	Xóa sạch nội dung cũ
os.O_EXCL	Lỗi nếu file đã tồn tại (dùng với O_CREATE)
os.O_SYNC	Ghi đồng bộ xuống disk ngay lập tức

3.4 Seek — Di chuyển con trỏ

```
f, _ := os.Open("data.bin")

f.Seek(10, io.SeekStart) // vị trí 10 từ đầu file
f.Seek(-5, io.SeekEnd)   // 5 bytes trước cuối
f.Seek(3, io.SeekCurrent) // 3 bytes từ vị trí hiện tại

// Lấy vị trí hiện tại
pos, _ := f.Seek(0, io.SeekCurrent)

// io.ReadAt — đọc tại offset mà KHÔNG di chuyển con trỏ (thread-safe)
buf := make([]byte, 100)
n, _ := f.ReadAt(buf, 512) // đọc 100 bytes tại offset 512

// Nhiều goroutine có thể ReadAt đồng thời
go f.ReadAt(buf1, 0)
go f.ReadAt(buf2, 512) // an toàn
```

3.5 Thao tác Filesystem

```
// Thông tin file
info, err := os.Stat("file.txt")
fmt.Println(info.Size(), info.ModTime(), info.Mode())

// Liệt kê thư mục
entries, _ := os.ReadDir(".")
for _, e := range entries {
    fmt.Println(e.Name(), e.IsDir(), e.Type())
}

// Tạo thư mục
os.Mkdir("newdir", 0755)
os.MkdirAll("a/b/c", 0755) // tạo cả cây

// File tạm
tmp, _ := os.CreateTemp("", "prefix-*.tmp")
defer os.Remove(tmp.Name())

// Rename, Remove, Symlink
os.Rename("old.txt", "new.txt")
os.Remove("file.txt")
os.RemoveAll("dir/") // xóa cả cây thư mục
os.Symlink("target", "link")
```

4. Package bufio — Buffered I/O

Đọc/ghi từng byte rất chậm vì mỗi lần là một syscall. bufio gom nhiều thao tác nhỏ lại thành một lần syscall duy nhất. Đây là một trong những package quan trọng nhất khi làm việc với I/O.

4.1 bufio.Scanner — Đọc từng dòng (cách phổ biến nhất)

```
f, _ := os.Open("file.txt")
defer f.Close()

scanner := bufio.NewScanner(f)

// Tăng buffer cho dòng dài (mặc định 64KB)
scanner.Buffer(make([]byte, 1024*1024), 1024*1024)

for scanner.Scan() {           // false khi hết hoặc lỗi
    line := scanner.Text()      // dòng hiện tại, không có \n
    // hoặc scanner.Bytes()     // trả []byte, zero-copy
    process(line)
}

// QUAN TRỌNG: Kiểm tra lỗi SAU vòng lặp
if err := scanner.Err(); err != nil {
    log.Fatal(err)
}
```

⚠ Vòng lặp kết thúc khi hết file HOẶC khi có lỗi. Phải kiểm tra scanner.Err() để phân biệt hai trường hợp này.

Split Functions

```
scanner.Split(bufio.ScanLines) // mặc định — theo dòng
scanner.Split(bufio.ScanWords) // theo từ (cách nhau bởi whitespace)
scanner.Split(bufio.ScanBytes) // theo từng byte
scanner.Split(bufio.ScanRunes) // theo rune UTF-8

// Custom SplitFunc — chia theo ký tự '|'
scanner.Split(func(data []byte, atEOF bool) (advance int, token []byte, err
error) {
    if atEOF && len(data) == 0 {
        return 0, nil, nil
    }
    if i := bytes.IndexByte(data, '|'); i >= 0 {
        return i + 1, data[:i], nil
    }
    if atEOF {
        return len(data), data, nil
    }
    return 0, nil, nil // cần thêm data
})
```

4.2 bufio.Reader

```
reader := bufio.NewReaderSize(f, 64*1024) // buffer 64KB

// Đọc đến delimiter (bao gồm cả delimiter)
line, err := reader.ReadString('\n')
line2, err := reader.ReadBytes('\n') // trả []byte

// Xem trước không tiêu thụ data
peek, err := reader.Peek(5) // xem 5 bytes tiếp theo

// Đọc từng rune UTF-8
r, size, err := reader.ReadRune()
reader.UnreadRune() // trả lại rune vừa đọc

// Đọc từng byte
b, err := reader.ReadByte()
reader.UnreadByte()

// Discard N bytes (nhanh hơn đọc rồi bỏ)
n, err := reader.Discard(1024)
```

4.3 bufio.Writer

```
writer := bufio.NewWriterSize(f, 64*1024) // buffer 64KB

writer.WriteString("Hello\n")
writer.WriteByte('\n')
writer.WriteRune('đ')
fmt.Fprintf(writer, "Value: %d\n", 42)

// QUAN TRỌNG: Phải Flush để đẩy buffer xuống writer gốc
if err := writer.Flush(); err != nil {
    log.Fatal(err)
}

// Available: số bytes còn trống trong buffer
// Buffered: số bytes đang chờ flush
fmt.Println(writer.Available(), writer.Buffered())
```

✓ Dùng `defer writer.Flush()` để đảm bảo luôn flush khi function kết thúc. Tuy nhiên nếu cần xử lý lỗi flush, không dùng defer.

5. Package io — Utilities

5.1 io.Copy và các biến thể

```
// Copy src -> dst, trả về số bytes và lỗi
n, err := io.Copy(dst, src)

// CopyN: copy tối đa N bytes
n, err = io.CopyN(dst, src, 1024*1024) // max 1MB

// CopyBuffer: dùng buffer do bạn cung cấp (tránh allocation)
buf := make([]byte, 32*1024)
n, err = io.CopyBuffer(dst, src, buf)

// io.Copy tự động dùng sendfile() nếu src là *os.File
// và dst là *net.TCPConn - zero-copy!
io.Copy(tcpConn, file)
```

5.2 Read Helpers

```
// ReadAll: đọc toàn bộ reader vào []byte
data, err := io.ReadAll(r)

// ReadFull: đọc ĐÚNG len(p) bytes hoặc trả lỗi
buf := make([]byte, 100)
n, err := io.ReadFull(r, buf) // lỗi nếu EOF trước khi đủ 100 bytes

// ReadAtLeast: đọc ít nhất min bytes
n, err = io.ReadAtLeast(r, buf, 50)
```

5.3 TeeReader — Đọc và sao chép đồng thời

```
// TeeReader: khi đọc từ tee, data cũng được ghi vào w
var spy bytes.Buffer
tee := io.TeeReader(httpResp.Body, &spy)

// Đọc và parse JSON - spy sẽ nhận bản sao để debug
json.NewDecoder(tee).Decode(&result)
log.Printf("raw response: %s", spy.String())

// Copy file và tính hash cùng lúc
h := sha256.New()
tee2 := io.TeeReader(srcFile, h)
io.Copy(dstFile, tee2)
fmt.Printf("SHA256: %x\n", h.Sum(nil))
```

5.4 MultiWriter và MultiReader

```
// MultiWriter: ghi vào nhiều writer cùng lúc
logFile, _ := os.OpenFile("app.log", os.O_APPEND|os.O_CREATE|os.O_WRONLY, 0644)
var buf bytes.Buffer
mw := io.MultiWriter(os.Stdout, logFile, &buf)
fmt.Fprintln(mw, "Log này xuất hiện ở cả 3 nơi")

// MultiReader: nối nhiều reader thành một
header := strings.NewReader("--- HEADER ---\n")
body, _ := os.Open("body.txt")
footer := strings.NewReader("--- FOOTER ---\n")
mr := io.MultiReader(header, body, footer)
io.Copy(os.Stdout, mr) // đọc theo thứ tự: header, body, footer
```

5.5 LimitReader và SectionReader

```
// LimitReader: chỉ cho phép đọc tối đa N bytes
limited := io.LimitReader(r, 1<<20) // max 1MB
data, _ := io.ReadAll(limited)

// SectionReader: đọc một đoạn [off, off+n) của ReaderAt
// Không ảnh hưởng đến con trỏ của ReaderAt gốc
section := io.NewSectionReader(f, 100, 500) // offset 100, đọc 500 bytes
data, _ = io.ReadAll(section)

// SectionReader implement ReadAt, Seek, Read
section.Seek(0, io.SeekStart)
section.ReadAt(buf, 50)
```

5.6 io.Pipe — Kết nối goroutines

```
pr, pw := io.Pipe()

// Producer goroutine
go func() {
    defer pw.Close() // LUÔN close khi xong
    fmt.Fprintln(pw, "line 1")
    fmt.Fprintln(pw, "line 2")
    // pw.CloseWithError(err) nếu có lỗi
}()

// Consumer trong goroutine hiện tại
sc := bufio.NewScanner(pr)
for sc.Scan() {
    fmt.Println(sc.Text())
}

// Use case: encode JSON và POST mà không cần buffer
pr2, pw2 := io.Pipe()
go func() {
    json.NewEncoder(pw2).Encode(payload)
    pw2.Close()
}()
http.Post(url, "application/json", pr2)
```

5.7 io.Discard

```
// io.Discard: Writer bỏ đi mọi thứ ghi vào (như /dev/null)

// Tính hash mà không cần lưu data
h := sha256.New()
io.Copy(io.MultiWriter(h, io.Discard), src)

// Drain HTTP response body (bắt buộc để tái sử dụng connection)
io.Copy(io.Discard, resp.Body)
resp.Body.Close()

// Đo tốc độ đọc
n, _ := io.Copy(io.Discard, src)
```

6. In-memory I/O — bytes và strings

6.1 bytes.Buffer

```
// bytes.Buffer implement io.Reader, io.Writer, io.ByteScanner, io.RuneScanner
var buf bytes.Buffer

// Ghi
buf.WriteString("Hello")
buf.Write([]byte(", "))
buf.WriteByte('G')
buf.WriteRune('ö')
fmt.Fprintf(&buf, "%s!", "world")

// Đọc
fmt.Println(buf.String()) // "Hello, Göworld!"
fmt.Println(buf.Len())   // số bytes còn lại chưa đọc
fmt.Println(buf.Cap())   // dung lượng hiện tại

data := buf.Bytes() // slice tham chiếu vào buffer (zero-copy)

// Đọc rồi xóa
b, _ := buf.ReadByte()
r, _, _ := buf.ReadRune()
line, _ := buf.ReadString('\n')

// Reset buffer để tái sử dụng
buf.Reset()

// bytes.NewBuffer — khởi tạo với data sẵn có
buf2 := bytes.NewBuffer(existingData)
buf2.Next(5) // skip 5 bytes
```

6.2 bytes.Reader và strings.Reader

```
// bytes.Reader: wrap []byte thành ReadSeeker
r := bytes.NewReader([]byte{0x01, 0x02, 0x03, 0x04})
r.Seek(2, io.SeekStart)
b, _ := r.ReadByte() // 0x03

// strings.Reader: wrap string thành ReadSeeker
sr := strings.NewReader("Hello, 世界")
sr.Seek(7, io.SeekStart)
ru, size, _ := sr.ReadRune() // '世', size=3

fmt.Println(sr.Size()) // tổng kích thước ban đầu
fmt.Println(sr.Len())  // bytes còn lại chưa đọc
```

6.3 strings.Builder — Ghi string hiệu quả

```
// strings.Builder: tối ưu hóa cho việc build string
// Tốt hơn += trong vòng lặp (tránh allocation liên tục)
var sb strings.Builder

for i := 0; i < 1000; i++ {
    fmt.Fprintf(&sb, "item %d\n", i)
}

result := sb.String() // zero-copy
sb.Reset()           // tái sử dụng

// Tương đương bytes.Buffer nhưng chỉ ghi, trả string
```

7. Standard I/O

7.1 os.Stdin, os.Stdout, os.Stderr

```
// os.Stdin/Stdout/Stderr là *os.File - implement Reader/Writer

// Đọc từ stdin từng dòng
scanner := bufio.NewScanner(os.Stdin)
for scanner.Scan() {
    fmt.Println("Input:", scanner.Text())
}

// Ghi có màu ra stderr
fmt.Fprintln(os.Stderr, "\033[31mError:\033[0m something went wrong")

// Redirect stdout sang file
old := os.Stdout
f, _ := os.Create("output.log")
os.Stdout = f
fmt.Println("This goes to file")
os.Stdout = old
```

7.2 fmt.Scan* — Đọc có định dạng

```
var name string
var age int

fmt.Scan(&name, &age)           // cách nhau bởi whitespace
fmt.Scanln(&name, &age)        // đến hết dòng
fmt.Scanf("%s %d", &name, &age) // theo format string

// Đọc từ bất kỳ Reader nào
fmt.Fscan(r, &a, &b, &c)
fmt.Fscanf(r, "%s %d", &name, &age)

// Đọc từ string
fmt.Sscan("Alice 30", &name, &age)
```


8. HTTP I/O

HTTP trong Go được xây dựng hoàn toàn trên `io.Reader`/`io.Writer`. `Request.Body` và `Response.Body` đều là `io.ReadCloser`; `ResponseWriter` implement `io.Writer` — cho phép dùng trực tiếp với `json.Encoder`, `io.Copy`, `fmt.Fprintf`, v.v.

8.1 HTTP Server — Cơ bản

```
// Handler đơn giản — w là io.Writer
func helloHandler(w http.ResponseWriter, r *http.Request) {
    w.Header().Set("Content-Type", "text/plain")
    w.WriteHeader(http.StatusOK)
    fmt.Fprintln(w, "Hello, World!") // ghi trực tiếp
}

// Đọc request body an toàn
func echoHandler(w http.ResponseWriter, r *http.Request) {
    defer r.Body.Close() // LUÔN close body

    // Giới hạn kích thước tránh bị tấn công
    limited := io.LimitReader(r.Body, 1<<20) // max 1MB
    body, err := io.ReadAll(limited)
    if err != nil {
        http.Error(w, "cannot read body", http.StatusBadRequest)
        return
    }
    w.Write(body)
}

// JSON streaming — không cần marshal ra []byte trung gian
func createUser(w http.ResponseWriter, r *http.Request) {
    var user User
    if err := json.NewDecoder(r.Body).Decode(&user); err != nil {
        http.Error(w, err.Error(), 400)
        return
    }
    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(201)
    json.NewEncoder(w).Encode(user) // encode thẳng vào ResponseWriter
}
```

8.2 HTTP Server — Stream file

```
func downloadHandler(w http.ResponseWriter, r *http.Request) {
    f, err := os.Open("largefile.zip")
    if err != nil { http.Error(w, "not found", 404); return }
    defer f.Close()

    stat, _ := f.Stat()
    w.Header().Set("Content-Type", "application/zip")
    w.Header().Set("Content-Disposition", `attachment; filename="file.zip"`)
}
```

```
// http.ServeContent tự xử lý Range, ETag, Last-Modified
http.ServeContent(w, r, "file.zip", stat.ModTime(), f)

// Hoặc đơn giản hơn:
// io.Copy(w, f)
}
```

8.3 HTTP Server Streaming — SSE

```
func sseHandler(w http.ResponseWriter, r *http.Request) {
    flusher, ok := w.(http.Flusher) // kiểm tra hỗ trợ flush
    if !ok {
        http.Error(w, "streaming unsupported", 500)
        return
    }

    w.Header().Set("Content-Type", "text/event-stream")
    w.Header().Set("Cache-Control", "no-cache")
    w.Header().Set("Connection", "keep-alive")

    ticker := time.NewTicker(time.Second)
    defer ticker.Stop()

    for {
        select {
        case t := <-ticker.C:
            fmt.Fprintf(w, "data: %s\n\n", t.Format(time.RFC3339))
            flusher.Flush() // đẩy về client ngay lập tức
        case <-r.Context().Done(): // client đóng kết nối
            return
        }
    }
}
```

8.4 HTTP Server — ResponseWriter Interface đầy đủ

Interface	Method	Khi nào dùng
http.ResponseWriter	Header() / Write() / WriteHeader()	Mặc định — mọi handler
http.Flusher	Flush()	Streaming response, SSE, chunked transfer
http.Hijacker	Hijack() (net.Conn, ...)	WebSocket — chiếm raw TCP connection
http.CloseNotifier	CloseNotify() <-chan bool	Deprecated — dùng r.Context().Done()
io.ReaderFrom	ReadFrom(r io.Reader)	Zero-copy sendfile khi serve static file

Wrap ResponseWriter — Middleware Pattern

```
// Capture status code và bytes written
type responseRecorder struct {
    http.ResponseWriter
    statusCode int
    bytesWritten int
}

func (r *responseRecorder) WriteHeader(code int) {
    r.statusCode = code
    r.ResponseWriter.WriteHeader(code)
}

func (r *responseRecorder) Write(b []byte) (int, error) {
    n, err := r.ResponseWriter.Write(b)
    r.bytesWritten += n
    return n, err
}

// Logging middleware
func loggingMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        rec := &responseRecorder{ResponseWriter: w, statusCode: 200}
        start := time.Now()
        next.ServeHTTP(rec, r)
        log.Printf("%s %s → %d (%d bytes) %v",
            r.Method, r.URL.Path, rec.statusCode,
            rec.bytesWritten, time.Since(start))
    })
}

// Gzip middleware
func gzipMiddleware(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        if !strings.Contains(r.Header.Get("Accept-Encoding"), "gzip") {
            next.ServeHTTP(w, r)
            return
        }
        gz := gzip.NewWriter(w)
        defer gz.Close()
        w.Header().Set("Content-Encoding", "gzip")
        next.ServeHTTP(gzipWriter{w, gz}, r)
    })
}

type gzipWriter struct { http.ResponseWriter; w io.Writer }
func (g gzipWriter) Write(b []byte) (int, error) { return g.w.Write(b) }
```

8.5 HTTP Client

```
// GET cơ bản
resp, err := http.Get("https://api.example.com/data")
if err != nil { log.Fatal(err) }
defer resp.Body.Close() // LUÔN close, dù không đọc

if resp.StatusCode != http.StatusOK {
    errBody, _ := io.ReadAll(io.LimitReader(resp.Body, 4096))
```

```
    log.Fatalf("status %d: %s", resp.StatusCode, errBody)
}

var result map[string]any
json.NewDecoder(resp.Body).Decode(&result)

// POST JSON – dùng io.Pipe để không buffer toàn bộ vào RAM
pr, pw := io.Pipe()
go func() {
    json.NewEncoder(pw).Encode(payload)
    pw.Close()
}()
resp, _ = http.Post(url, "application/json", pr)

// HTTP Client với timeout và custom transport
client := &http.Client{
    Timeout: 30 * time.Second,
    Transport: &http.Transport{
        MaxIdleConns:    100,
        MaxIdleConnsPerHost: 10,
        IdleConnTimeout: 90 * time.Second,
    },
}
```

8.6 HTTP Client — Upload File

```
// Upload file nhỏ – buffer toàn bộ vào RAM
func uploadFile(path string) error {
    var buf bytes.Buffer
    mw := multipart.NewWriter(&buf)
    mw.WriteField("description", "my file")
    part, _ := mw.CreateFormFile("file", filepath.Base(path))
    f, _ := os.Open(path)
    defer f.Close()
    io.Copy(part, f)
    mw.Close()
    resp, err := http.Post(url, mw.FormDataContentType(), &buf)
    defer resp.Body.Close()
    return err
}

// Upload file lớn – streaming, không buffer vào RAM
func uploadLargeFile(path string) error {
    pr, pw := io.Pipe()
    mw := multipart.NewWriter(pw)
    go func() {
        defer pw.Close()
        part, _ := mw.CreateFormFile("file", filepath.Base(path))
        f, _ := os.Open(path); defer f.Close()
        io.Copy(part, f)
        mw.Close()
    }()
    resp, err := http.Post(url, mw.FormDataContentType(), pr)
    if err != nil { return err }
    defer resp.Body.Close()
    return nil
}
```


9. Network I/O — Package net

9.1 TCP Server

```
func tcpServer() {
    ln, err := net.Listen("tcp", ":8080")
    if err != nil { log.Fatal(err) }
    defer ln.Close()

    for {
        conn, err := ln.Accept()
        if err != nil { log.Println(err); continue }
        go handleConn(conn) // mỗi connection một goroutine
    }
}

func handleConn(conn net.Conn) {
    defer conn.Close()

    // LUÔN set deadline tránh goroutine treo vĩnh viễn
    conn.SetDeadline(time.Now().Add(30 * time.Second))

    reader := bufio.NewReader(conn)
    for {
        line, err := reader.ReadString('\n')
        if err != nil {
            if err == io.EOF { return }
            if netErr, ok := err.(net.Error); ok && netErr.Timeout() {
                log.Println("connection timeout")
            }
            return
        }
        // Reset deadline sau mỗi lần nhận data
        conn.SetDeadline(time.Now().Add(30 * time.Second))
        conn.Write([]byte(strings.ToUpper(line)))
    }
}
```

9.2 TCP Client

```
conn, err := net.Dial("tcp", "localhost:8080")
if err != nil { log.Fatal(err) }
defer conn.Close()

conn.SetDeadline(time.Now().Add(10 * time.Second))

// Ghi vào conn như ghi vào bất kỳ Writer nào
fmt.Fprintln(conn, "hello server")

// Đọc response
response, _ := bufio.NewReader(conn).ReadString('\n')
fmt.Print(response)

// DialTimeout - timeout khi connect
```

```
conn2, err := net.DialTimeout("tcp", "host:port", 5*time.Second)
```

9.3 UDP, Unix Socket

```
// UDP Server
conn, _ := net.ListenPacket("udp", ":9090")
defer conn.Close()
buf := make([]byte, 1024)
for {
    n, addr, _ := conn.ReadFrom(buf)
    fmt.Printf("from %s: %s\n", addr, buf[:n])
    conn.WriteTo([]byte("pong"), addr)
}

// UDP Client
conn2, _ := net.Dial("udp", "localhost:9090")
conn2.Write([]byte("ping"))

// Unix Domain Socket
os.Remove("/tmp/app.sock")
ln, _ := net.Listen("unix", "/tmp/app.sock")
// client:
conn3, _ := net.Dial("unix", "/tmp/app.sock")
```

9.4 net.Error — Xử lý lỗi network

```
if netErr, ok := err.(net.Error); ok {
    if netErr.Timeout() {
        // Timeout - có thể retry
        log.Println("timeout, retrying...")
        conn.SetDeadline(time.Now().Add(30 * time.Second))
    }
    if netErr.Temporary() {
        // Lỗi tạm thời (e.g. EAGAIN) - retry sau chút
        time.Sleep(time.Millisecond * 100)
    }
}
```

10. Encoding I/O Streams

Tất cả encoder/decoder trong Go đều nhận `io.Reader`/`io.Writer`. Không bao giờ cần `marshal/unmarshal` vào `[]byte` trung gian khi có thể encode/decode trực tiếp từ stream.

10.1 JSON Streaming

```
// Encode stream - không cần marshal ra []byte
enc := json.NewEncoder(w) // w là bất kỳ io.Writer
enc.SetIndent("", " ")    // pretty print
enc.SetEscapeHTML(false)  // tắt HTML escaping
enc.Encode(data)

// Decode stream
dec := json.NewDecoder(r) // r là bất kỳ io.Reader

// Decode nhiều JSON objects liên tiếp
for dec.More() { // còn dữ liệu?
    var obj MyStruct
    if err := dec.Decode(&obj); err != nil { break }
    process(obj)
}

// Stream JSON array lớn từ file - không load cả file vào RAM
f, _ := os.Open("large.json")
dec2 := json.NewDecoder(f)
dec2.Token() // bỏ qua dấu '[' mở đầu
for dec2.More() {
    var item Item
    dec2.Decode(&item)
    process(item)
}
dec2.Token() // dấu ']' kết thúc

// UseNumber để tránh float64 precision loss
dec3 := json.NewDecoder(r)
dec3.UseNumber()
dec3.DisallowUnknownFields() // strict mode
```

10.2 CSV Streaming

```
// Đọc CSV từng record
r := csv.NewReader(file)
r.LazyQuotes = true
r.TrimLeadingSpace = true
r.Comment = '#'
r.FieldsPerRecord = -1 // -1: số field không cố định

for {
    record, err := r.Read()
    if err == io.EOF { break }
    if err != nil { log.Fatal(err) }
```

```
    fmt.Println(record)
}

// Ghi CSV
w := csv.NewWriter(os.Stdout)
w.Comma = ';'
w.Write([]string{"name", "age", "email"})
w.WriteAll(records) // Flush tự động
if err := w.Error(); err != nil { log.Fatal(err) }
```

10.3 XML Streaming

```
// XML decoder - xử lý từng token
dec := xml.NewDecoder(r)
dec.Strict = false
for {
    tok, err := dec.Token()
    if err == io.EOF { break }
    switch t := tok.(type) {
    case xml.StartElement:
        if t.Name.Local == "item" {
            var item Item
            dec.DecodeElement(&item, &t)
            process(item)
        }
    case xml.CharData:
        fmt.Println("text:", string(t))
    }
}

// XML encoder
enc := xml.NewEncoder(w)
enc.Indent("", " ")
enc.Encode(data)
```

10.4 Gzip / Compression

```
// Nén khi ghi
gw, _ := gzip.NewWriterLevel(file, gzip.BestCompression)
defer gw.Close()
json.NewEncoder(gw).Encode(data) // JSON → gzip → file

// Giải nén khi đọc
gr, err := gzip.NewReader(file)
if err != nil { log.Fatal(err) }
defer gr.Close()
json.NewDecoder(gr).Decode(&data) // file → gunzip → JSON

// Pipeline nén nhiều tầng
gz := gzip.NewWriter(file)
bw := bufio.NewWriter(gz) // buffer trước khi nén
enc := json.NewEncoder(bw) // JSON → buffer → gzip → file
enc.Encode(data)
bw.Flush()
```

```
gz.Close()

// zlib, deflate - cùng pattern
zw := zlib.NewWriter(w)
zr, _ := zlib.NewReader(r)
```

10.5 Base64

```
// Encode base64 khi ghi
encoder := base64.NewEncoder(base64.StdEncoding, w)
encoder.Write(binaryData)
encoder.Close() // flush padding

// Decode base64 khi đọc
decoder := base64.NewDecoder(base64.StdEncoding, r)
data, _ := io.ReadAll(decoder)

// URL-safe base64 (không dùng +/=)
enc2 := base64.NewEncoder(base64.URLEncoding, w)
```

10.6 Hash — Tính checksum

```
// Tính SHA256 khi đọc file - không cần đọc 2 lần
f, _ := os.Open("file.zip")
h := sha256.New()
io.Copy(io.MultiWriter(destination, h), f)
fmt.Printf("SHA256: %x\n", h.Sum(nil))

// hash.Hash implement io.Writer
h2 := sha256.New()
fmt.Fprintf(h2, "data to hash")
h2.Write(moreData)
fmt.Printf("%x\n", h2.Sum(nil))

// MD5, SHA1, SHA512 - cùng interface
h3 := md5.New()
h4 := sha1.New()
h5 := sha512.New()
```

10.7 Binary I/O

```
import "encoding/binary"

// Đọc struct binary
var header struct {
    Magic    [4]byte
    Version  uint32
    Length  uint64
}
binary.Read(f, binary.LittleEndian, &header)
```



```
// Ghi struct binary
binary.Write(f, binary.BigEndian, &header)

// Đọc/ghi từng giá trị
var n uint32
binary.Read(r, binary.LittleEndian, &n)
binary.Write(w, binary.LittleEndian, uint32(42))

// Varint - encoding tiết kiệm cho số nhỏ
buf := make([]byte, binary.MaxVarintLen64)
n2 := binary.PutVarint(buf, 12345)
val, _ := binary.ReadVarint(bytes.NewReader(buf[:n2]))
```

11. Tự Implement Interface I/O

Đây là kỹ năng quan trọng nhất. Bằng cách implement `io.Reader` hoặc `io.Writer`, bạn tạo ra các component có thể kết hợp với toàn bộ hệ sinh thái I/O của Go.

11.1 Custom Reader

```
// Infinite zero reader (như /dev/zero)
type ZeroReader struct{}
func (ZeroReader) Read(p []byte) (int, error) {
    for i := range p { p[i] = 0 }
    return len(p), nil
}
io.CopyN(dst, ZeroReader{}, 1024*1024) // 1MB zeros

// Progress reader - báo cáo tiến độ
type ProgressReader struct {
    r      io.Reader
    total  int64
    read   int64
    onUpdate func(pct float64)
}

func (pr *ProgressReader) Read(p []byte) (int, error) {
    n, err := pr.r.Read(p)
    pr.read += int64(n)
    if pr.total > 0 {
        pr.onUpdate(float64(pr.read) / float64(pr.total) * 100)
    }
    return n, err
}

// Dùng:
f, _ := os.Open("largefile.iso")
stat, _ := f.Stat()
pr := &ProgressReader{
    r: f, total: stat.Size(),
    onUpdate: func(pct float64) { fmt.Printf("\r%.1f%%", pct) },
}
io.Copy(destination, pr)
```

11.2 Custom Writer

```
// Counting writer - đếm số bytes đã ghi
type CountingWriter struct {
    w      io.Writer
    total  int64
}

func (cw *CountingWriter) Write(p []byte) (int, error) {
    n, err := cw.w.Write(p)
    cw.total += int64(n)
}
```

```
    return n, err
}

cw := &CountingWriter{w: os.Stdout}
fmt.Fprintln(cw, "hello world")
fmt.Println("Wrote:", cw.total, "bytes")

// Throttle writer - giới hạn tốc độ ghi
type ThrottleWriter struct {
    w      io.Writer
    rate   int           // bytes per second
    written int
    start  time.Time
}

func (t *ThrottleWriter) Write(p []byte) (int, error) {
    elapsed := time.Since(t.start).Seconds()
    allowed := int(elapsed*float64(t.rate)) - t.written
    if allowed <= 0 {
        time.Sleep(time.Millisecond * 10)
        return 0, nil
    }
    if allowed > len(p) { allowed = len(p) }
    n, err := t.w.Write(p[:allowed])
    t.written += n
    return n, err
}
```

11.3 Retry Reader

```
type RetryReader struct {
    newReader func() (io.ReadCloser, error)
    current   io.ReadCloser
    attempts  int
    maxRetry  int
}

func (r *RetryReader) Read(p []byte) (int, error) {
    for {
        if r.current == nil {
            rc, err := r.newReader()
            if err != nil { return 0, err }
            r.current = rc
        }
        n, err := r.current.Read(p)
        if err != nil && err != io.EOF && r.attempts < r.maxRetry {
            r.current.Close()
            r.current = nil
            r.attempts++
            continue
        }
        return n, err
    }
}
```

12. Concurrent I/O

12.1 Đọc nhiều file song song

```
func readConcurrent(files []string) ([][]byte, error) {
    results := make([][]byte, len(files))
    errs    := make([]error, len(files))

    var wg sync.WaitGroup
    for i, path := range files {
        wg.Add(1)
        go func(i int, path string) {
            defer wg.Done()
            results[i], errs[i] = os.ReadFile(path)
        }(i, path)
    }
    wg.Wait()

    for _, err := range errs {
        if err != nil { return nil, err }
    }
    return results, nil
}
```

12.2 Fan-out Pipeline

```
// Đọc 1 Reader, phân phối cho nhiều goroutine xử lý song song
func fanOut(r io.Reader, workers int) {
    lines := make(chan string, 100)

    // Producer
    go func() {
        defer close(lines)
        sc := bufio.NewScanner(r)
        for sc.Scan() { lines <- sc.Text() }
    }()

    // Consumers
    var wg sync.WaitGroup
    for i := 0; i < workers; i++ {
        wg.Add(1)
        go func() {
            defer wg.Done()
            for line := range lines { process(line) }
        }()
    }
    wg.Wait()
}
```

13. Testing với I/O

13.1 bytes.Buffer và strings.NewReader

```
// Test Writer - dùng bytes.Buffer thay file thật
func TestMyWriter(t *testing.T) {
    var buf bytes.Buffer
    myFunc(&buf)
    if got := buf.String(); got != "expected output\n" {
        t.Errorf("got %q", got)
    }
}

// Test Reader - dùng strings.NewReader thay file thật
func TestMyReader(t *testing.T) {
    r := strings.NewReader("line1\nline2\nline3\n")
    result := processLines(r)
    // assert...
}
```

13.2 httptest — Test HTTP không cần server thật

```
func TestCreateUser(t *testing.T) {
    body := strings.NewReader(`{"name":"Alice","email":"alice@example.com"}`)
    req := httptest.NewRequest(http.MethodPost, "/api/user", body)
    req.Header.Set("Content-Type", "application/json")

    rec := httptest.NewRecorder() // implement ResponseWriter
    createUserHandler(rec, req)

    res := rec.Result()
    defer res.Body.Close()

    if res.StatusCode != http.StatusCreated {
        t.Errorf("expected 201, got %d", res.StatusCode)
    }

    var user User
    json.NewDecoder(res.Body).Decode(&user)
    if user.Name != "Alice" {
        t.Errorf("expected Alice, got %s", user.Name)
    }
}

// httptest.Server - test client code
func TestClient(t *testing.T) {
    ts := httptest.NewServer(http.HandlerFunc(func(w http.ResponseWriter, r
    *http.Request) {
        fmt.Fprintln(w, `{"status":"ok"}`)
    }))
    defer ts.Close()

    resp, _ := http.Get(ts.URL + "/api")
    defer resp.Body.Close()
}
```

```
    // assert...  
}
```

13.3 io.Pipe trong Test

```
func TestStreaming(t *testing.T) {  
    pr, pw := io.Pipe()  
  
    go func() {  
        writeStream(pw) // hàm ghi streaming  
        pw.Close()  
    }()  
  
    sc := bufio.NewScanner(pr)  
    var lines []string  
    for sc.Scan() {  
        lines = append(lines, sc.Text())  
    }  
    // assert lines...  
}
```

14. Xử lý Lỗi I/O

14.1 Các sentinel errors

Error	Ý nghĩa	Cách xử lý
io.EOF	Hết dữ liệu — KHÔNG phải lỗi	Kết thúc vòng lặp đọc bình thường
io.ErrUnexpectedEOF	EOF giữa chừng khi đọc dữ liệu có cấu trúc	Dữ liệu bị cắt — báo lỗi
io.ErrClosedPipe	Ghi vào pipe đã đóng	Kiểm tra thứ tự đóng pipe
io.ErrNoProgress	Reader trả 0 bytes quá nhiều lần	Dữ liệu bị kẹt — abort
io.ErrShortBuffer	Buffer nhỏ hơn dữ liệu cần đọc	Dùng buffer lớn hơn
io.ErrShortWrite	Write không ghi hết dữ liệu	Retry phần còn lại
fs.ErrNotExist	File/directory không tồn tại	Kiểm tra trước hoặc xử lý
fs.ErrPermission	Không có quyền truy cập	Kiểm tra quyền file
syscall.ENOSPC	Disk đầy	Dọn disk hoặc thông báo lỗi

14.2 Pattern xử lý lỗi I/O

```
// Phân biệt lỗi bình thường với lỗi thật
for {
    n, err := r.Read(buf)
    if n > 0 {
        process(buf[:n]) // xử lý data trước
    }
    if err == io.EOF { break }           // hết data - bình thường
    if err != nil { return err }         // lỗi thật
}

// Kiểm tra lỗi hệ thống
if errors.Is(err, fs.ErrNotExist) { /* file không tồn tại */ }
if errors.Is(err, fs.ErrPermission) { /* không có quyền */ }
if errors.Is(err, syscall.ENOSPC) { /* disk đầy */ }

// Xử lý lỗi network
if netErr, ok := err.(net.Error); ok {
    if netErr.Timeout() { /* timeout - có thể retry */ }
    if netErr.Temporary() { /* tạm thời - retry sau */ }
}

// Unwrap path error
var pathErr *fs.PathError
if errors.As(err, &pathErr) {
    fmt.Println("op:", pathErr.Op, "path:", pathErr.Path)
}
```

15. Patterns Thực Tế

15.1 Xử lý file lớn từng dòng

```
func processLargeFile(path string) error {
    f, err := os.Open(path)
    if err != nil { return err }
    defer f.Close()

    scanner := bufio.NewScanner(f)
    scanner.Buffer(make([]byte, 1024*1024), 1024*1024) // 1MB cho dòng dài

    lineNum := 0
    for scanner.Scan() {
        lineNum++
        if err := process(scanner.Text()); err != nil {
            return fmt.Errorf("line %d: %w", lineNum, err)
        }
    }
    return scanner.Err()
}
```

15.2 Fast Logger với buffer

```
type FastLogger struct {
    f *os.File
    w *bufio.Writer
    mu sync.Mutex
}

func NewFastLogger(path string) (*FastLogger, error) {
    f, err := os.OpenFile(path, os.O_APPEND|os.O_CREATE|os.O_WRONLY, 0644)
    if err != nil { return nil, err }
    return &FastLogger{f: f, w: bufio.NewWriterSize(f, 64*1024)}, nil
}

func (l *FastLogger) Log(msg string) {
    l.mu.Lock()
    defer l.mu.Unlock()
    fmt.Fprintf(l.w, "%s %s\n", time.Now().Format(time.RFC3339), msg)
}

func (l *FastLogger) Close() error {
    l.mu.Lock()
    defer l.mu.Unlock()
    l.w.Flush()
    return l.f.Close()
}
```

15.3 Drain HTTP Response Body


```
// LUÔN drain và close body để tái sử dụng TCP connection
resp, err := client.Do(req)
if err != nil { return err }
defer func() {
    io.Copy(io.Discard, resp.Body) // drain
    resp.Body.Close()
}()
```

15.4 Graceful Stream với Context

```
func copyWithContext(ctx context.Context, dst io.Writer, src io.Reader) error {
    buf := make([]byte, 32*1024)
    for {
        select {
        case <-ctx.Done():
            return ctx.Err()
        default:
        }

        n, err := src.Read(buf)
        if n > 0 {
            if _, werr := dst.Write(buf[:n]); werr != nil {
                return werr
            }
        }
        if err == io.EOF { return nil }
        if err != nil { return err }
    }
}
```

16. Bảng Tóm Tắt — Chọn Công Cụ Đúng

Tình huống	Giải pháp tốt nhất
Đọc toàn bộ file nhỏ	<code>os.ReadFile()</code>
Ghi toàn bộ file	<code>os.WriteFile()</code>
Đọc file lớn từng dòng	<code>bufio.Scanner</code> với <code>scanner.Buffer()</code> lớn
Ghi log hiệu quả	<code>bufio.NewWriter</code> + <code>defer Flush()</code>
Copy giữa hai stream	<code>io.Copy(dst, src)</code>
Copy và tính hash cùng lúc	<code>io.TeeReader(src, hash)</code>
Ghi vào nhiều nơi cùng lúc	<code>io.MultiWriter(w1, w2, w3)</code>
Nối nhiều nguồn thành một	<code>io.MultiReader(r1, r2, r3)</code>
Giới hạn số bytes đọc	<code>io.LimitReader(r, maxBytes)</code>
In-memory buffer đọc/ghi	<code>bytes.Buffer</code>
Wrap string thành Reader	<code>strings.NewReader(s)</code>
Kết nối goroutines	<code>io.Pipe()</code>
Build string hiệu quả	<code>strings.Builder</code>
Test HTTP handler	<code>httptest.NewRecorder()</code>
Test HTTP client	<code>httptest.NewServer()</code>
JSON stream lớn	<code>json.NewDecoder(r).Decode()</code> trong vòng lặp
Nén/giải nén	<code>gzip.NewWriter(w)</code> / <code>gzip.NewReader(r)</code>
Tính checksum file	<code>sha256.New()</code> + <code>io.Copy(io.MultiWriter(dst, h), src)</code>
Binary protocol	<code>encoding/binary.Read/Write</code>
HTTP streaming response	<code>http.Flusher</code> + <code>w.Flush()</code>
Upload file lớn không buffer	<code>io.Pipe()</code> + <code>multipart.Writer</code>
Đọc nhiều file song song	<code>sync.WaitGroup</code> + goroutines
Drain HTTP body	<code>io.Copy(io.Discard, resp.Body)</code>
Embed file vào binary	<code>//go:embed</code> + <code>embed.FS</code>
Random access file	<code>f.ReadAt()</code> / <code>f.WriteAt()</code> — thread-safe

17. Quy Tắc Vàng

- ✓ Luôn defer `f.Close()` ngay sau khi `Open` — tránh rò rỉ file descriptor.
- ✓ Luôn drain và close HTTP response body dù không đọc — `io.Copy(io.Discard, resp.Body)`.
- ✓ Dùng `bufio` bất cứ khi nào đọc/ghi nhiều lần nhỏ — tránh quá nhiều syscall.
- ✓ Kiểm tra `scanner.Err()` sau vòng lặp `scanner.Scan()` — vòng lặp kết thúc không có nghĩa là thành công.
- ✓ Dùng `io.LimitReader` khi đọc input từ bên ngoài — tránh bị tấn công bằng dữ liệu lớn.
- ✓ Prefer `io.Copy` thay vì tự đọc rồi ghi — tự động zero-copy khi có thể.
- ✓ Dùng `json.NewEncoder/Decoder` thay vì `json.Marshal/Unmarshal` khi có stream — tránh buffer trung gian.
- ⚠ Đọc phải xử lý $n > 0$ trước khi kiểm tra `err` — một lần `Read` có thể trả về cả data lẫn `io.EOF`.
- ⚠ `net.Conn` LUÔN phải có `SetDeadline` — không có deadline, goroutine có thể treo vĩnh viễn khi client mất kết nối.
- ⚠ `bufio.Writer` PHẢI Flush trước khi `Close` — `Close` không Flush tự động.

Triết lý cốt lõi của I/O trong Go

Toàn bộ hệ thống được xây dựng trên nguyên tắc *composition* — bạn không cần biết data đến từ đâu hay đi về đâu. Một hàm nhận `io.Writer` hoạt động với `os.File`, `net.Conn`, `http.ResponseWriter`, `bytes.Buffer`, hay bất kỳ custom writer nào. `io.Copy` chỉ vài dòng code nhưng có thể copy file, stream network, response HTTP, hay kết nối hai goroutine qua pipe — tất cả đều giống nhau.